

Guía implementación Redis Cache en SpringBoot

Introducción

Redis (Remote Dictionary Server) es una base de datos en memoria, de tipo clave-valor, altamente eficiente y de código abierto, diseñada para ofrecer operaciones extremadamente rápidas. Gracias a su soporte para estructuras de datos avanzadas, persistencia opcional y compatibilidad con múltiples lenguajes de programación, Redis se ha convertido en una de las soluciones más populares para la gestión de caché en entornos de producción.

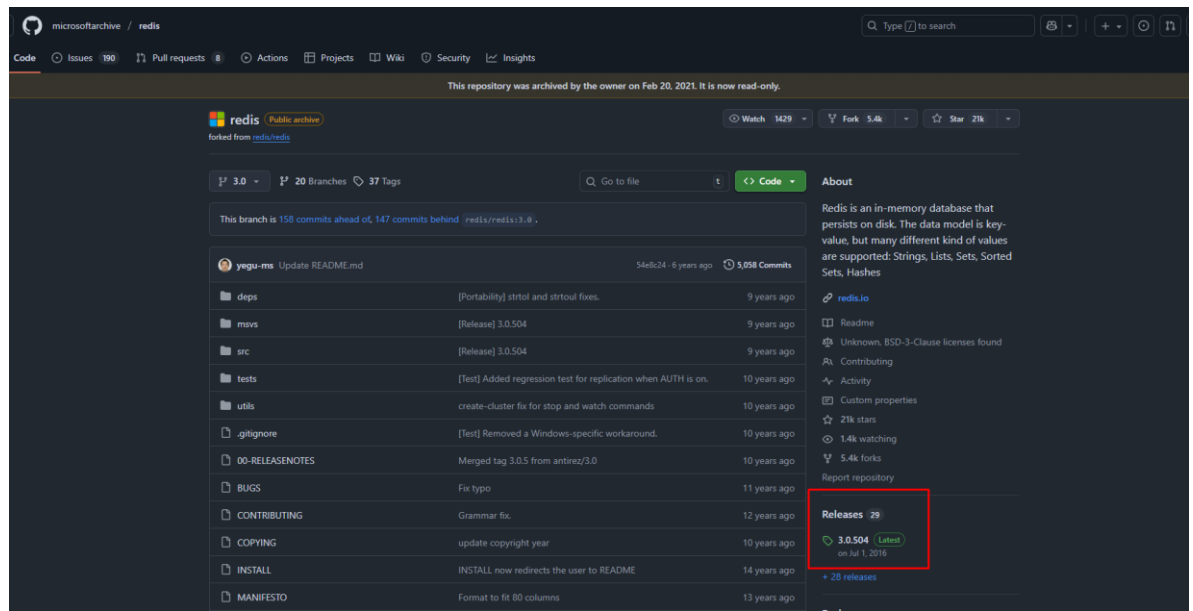
Por su parte, Spring Boot, uno de los frameworks más utilizados en el ecosistema Java, ofrece una integración nativa y sencilla con Redis a través del módulo Spring Data Redis, permitiendo a los desarrolladores configurar y administrar el almacenamiento en caché con anotaciones simples y una mínima configuración adicional.

Esta guía tiene como propósito describir de forma detallada el proceso de descarga, instalación en Windows, configuración e integración de Redis Cache en un proyecto Spring Boot, incluyendo la creación de configuraciones personalizadas de expiración de caché y ejemplos prácticos de uso.

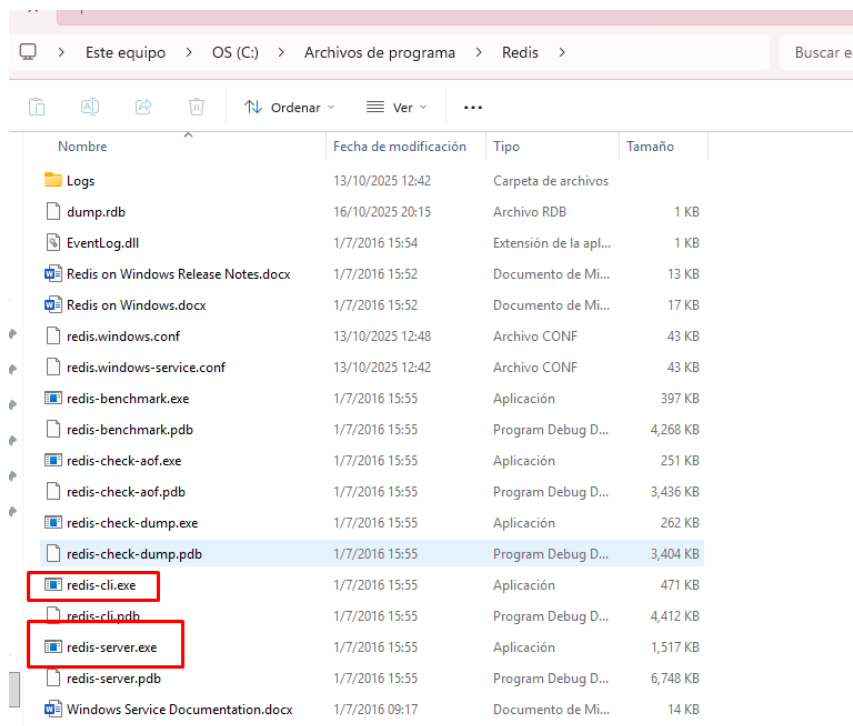
Pasos para integración de Redis Cache:

- Instalación del servicio Redis Cache en Windows :
 - Dirigirse al repositorio oficial de Windows archives y descargar el .zip o instalador que desee

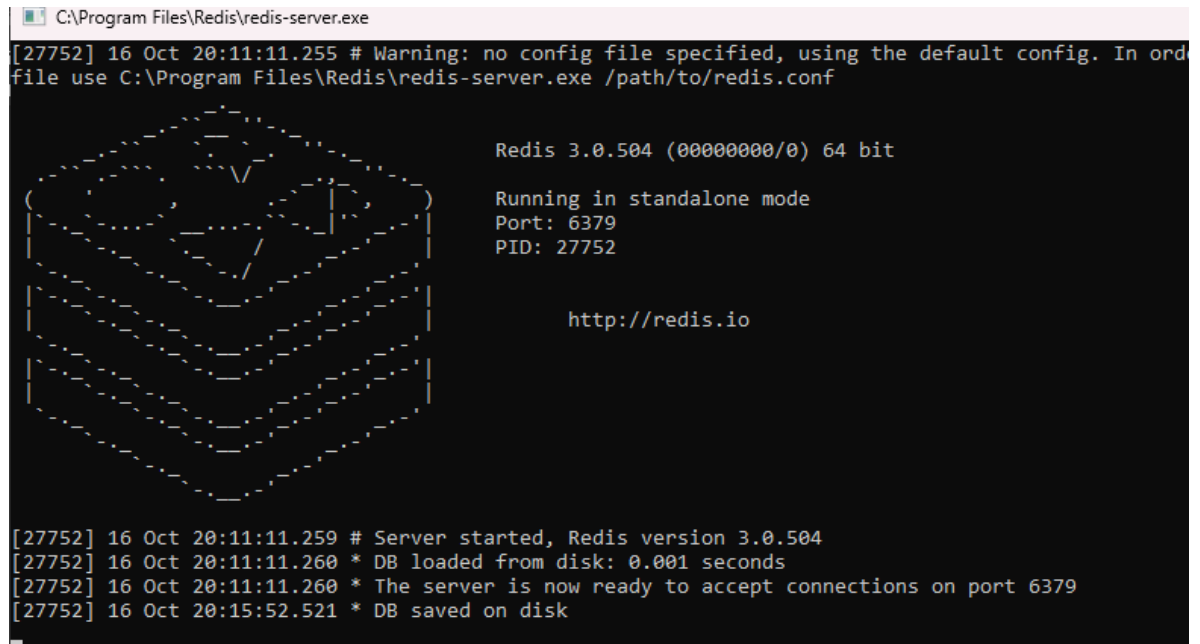
[Link de descarga oficial](#)



- Ejecución del servicio Redis :
 - Ejecutar el archivo redis-server.exe ubicado en la carpeta Redis que se instalo en el paso anterior.
 - Ejecutar el archivo redis-cli.exe ubicado en la carpeta Redis que se instalo en el paso anterior.



- Al ejecutar el servicio, se mostrará una pantalla de confirmación indicando que este se ha iniciado correctamente y que está operativo en el puerto correspondiente.



```
C:\Program Files\Redis\redis-server.exe
[27752] 16 Oct 20:11:11.255 # Warning: no config file specified, using the default config. In order to use a config file use C:\Program Files\Redis\redis-server.exe /path/to/redis.conf

  _____
 /                \
|                    |
|      V            |
|                    |
 \                /
  _____

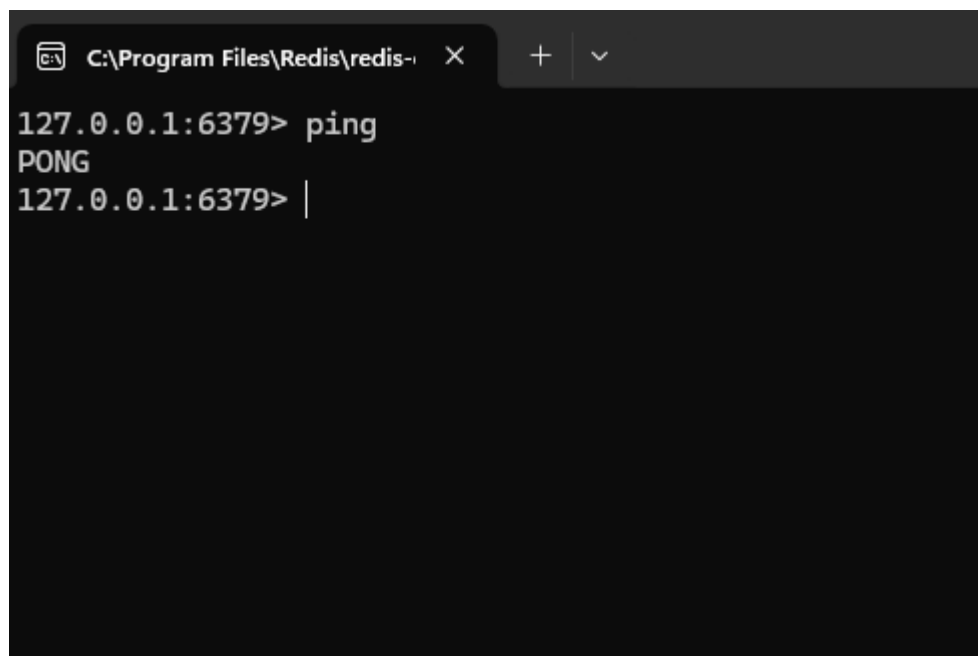
Redis 3.0.504 (00000000/0) 64 bit

Running in standalone mode
Port: 6379
PID: 27752

http://redis.io

[27752] 16 Oct 20:11:11.259 # Server started, Redis version 3.0.504
[27752] 16 Oct 20:11:11.260 * DB loaded from disk: 0.001 seconds
[27752] 16 Oct 20:11:11.260 * The server is now ready to accept connections on port 6379
[27752] 16 Oct 20:15:52.521 * DB saved on disk
```

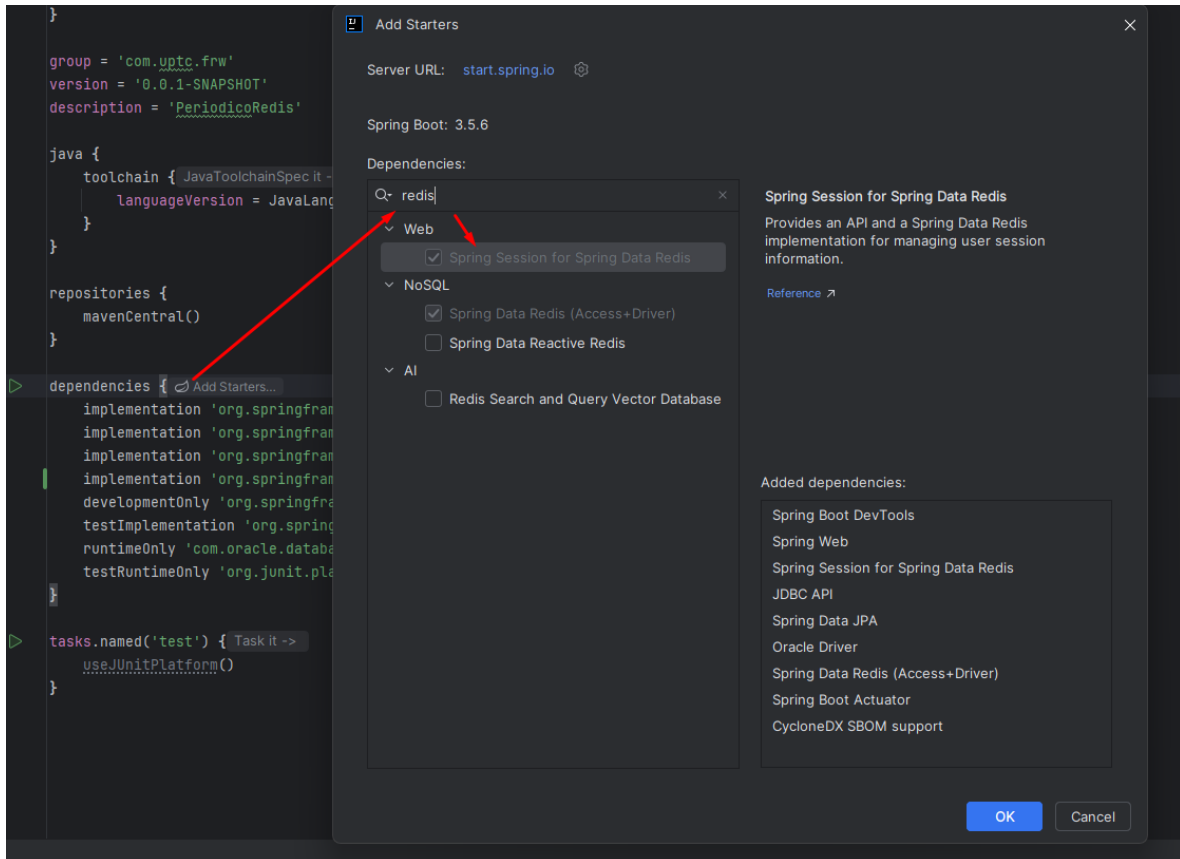
- En la pantalla de redis cli podrán hacer un ping para confirmar el funcionamiento y respuesta del servicio



```
C:\Program Files\Redis\redis-
127.0.0.1:6379> ping
PONG
127.0.0.1:6379> |
```

- En el proyecto SpringBoot se deberá de implementar la librería data Redis, en este caso usamos gradle, por tal motivo nos dirigimos a nuestro archivo gradle.build.

```
a. implementation 'org.springframework.session:spring-session-data-redis'
b. implementation 'org.springframework.boot:spring-boot-starter-data-redis'
```



```
1 dependencies { Add Starters...
2     implementation 'org.springframework.boot:spring-boot-starter-web'
3     implementation 'org.springframework.boot:spring-boot-starter-actuator'
4     implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
5     implementation 'org.springframework.session:spring-session-data-redis'
6     developmentOnly 'org.springframework.boot:spring-boot-devtools'
7     implementation 'org.springframework.boot:spring-boot-starter-data-redis'
8     testImplementation 'org.springframework.boot:spring-boot-starter-test'
9     runtimeOnly 'com.oracle.database.jdbc:ojdbc11'
10    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
11 }
12
13 tasks.named('test') { Task it ->
```

- Luego de instalar las dependencias se deberá de configurar el archivo application.properties de nuestro proyecto indicándole el host donde está el servicio de redis cache ejecutándose y el puerto .

```
spring.application.name=PeriodicoRedis
server.port=8081
spring.datasource.url=jdbc:oracle:thin::prd900
spring.datasource.username=PERIODICO
spring.datasource.password=A
spring.datasource.driver-class-name=oracle.jdbc.OracleDriver

spring.data.redis.host=localhost
spring.data.redis.port=6379
```

- Crear archivo de configuración de Redis en el que se asignara el tiempo por defecto en el que guardara los datos en cache.

```
@Configuration
@EnableCaching
public class Config {

    @Bean
    public RedisCacheConfiguration cacheConfiguration() {
        return RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(5))
            .serializeValuesWith(
                RedisSerializationContext.SerializationPair.fromSerializer(
                    new
                    GenericJackson2JsonRedisSerializer()
                )
            );
    }

    @Bean
    public RedisCacheManagerBuilderCustomizer
    redisCacheManagerBuilderCustomizer() {
        return (builder) -> builder
            .withCacheConfiguration("agencyCache",
                RedisCacheConfiguration.defaultCacheConfig()
                    .entryTtl(Duration.ofMinutes(5))
                    .serializeValuesWith(
                        RedisSerializationContext.SerializationPair.fromSerializer(
                            new
                            GenericJackson2JsonRedisSerializer()
                        )
                    )
            )
            .withCacheConfiguration("agencyByIdCache",
```

```

        RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofSeconds(10))
            .serializeValuesWith(

RedisSerializationContext.SerializationPair.fromSerializer(
                                new
GenericJackson2JsonRedisSerializer()
                                )
            ))
        .withCacheConfiguration("newCache",
            RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(5))
                .serializeValuesWith(

RedisSerializationContext.SerializationPair.fromSerializer(
                                new
GenericJackson2JsonRedisSerializer()
                                )
            ))
        .withCacheConfiguration("newByIdCache",
            RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofSeconds(10))
                .serializeValuesWith(

RedisSerializationContext.SerializationPair.fromSerializer(
                                new
GenericJackson2JsonRedisSerializer()
                                )
            ));
    }
}

```

- Agregar la etiqueta @Cacheable en los endpoint que queremos que guarde los datos, indicándole el valor y la key que guardara.

```
Config.java  AgencyService.java  application.properties  build.gradle (PeriodicoRedis)

1 package com.uptc.frw.periodicoredis.service;
2
3
4 import com.uptc.frw.periodicoredis.model.Agency;
5 import com.uptc.frw.periodicoredis.repository.AgencyRepository;
6 import org.springframework.beans.factory.annotation.Autowired;
7 import org.springframework.cache.annotation.Cacheable;
8 import org.springframework.stereotype.Service;
9
10 import java.util.List;
11
12 @Service 2 usages jonathan *
13 public class AgencyService {
14     @Autowired
15     private AgencyRepository agencyRepository;
16
17     @Cacheable(value = "agencyCache", key = "agencyCache") 1 usage jonathan
18     public List<Agency> findAllAgency() { return agencyRepository.findAll(); }
19
20
21
22     @Cacheable(value = "agencyByIdCache", key = "agencyByIdCache") 2 usages jonathan
23     public Agency getAgencyById(Long id) {
24         Agency agency = agencyRepository.findById(id).orElse( other: null);
25         return agency;
26     }
27 }
```

- Iniciar servicios y validar el guardado de las consultas en el redis insight (descargar herramienta) con el comando SCAN 0 el cual mostrara las llaves que se tienen activas.

SCAN 0

- 1) "0"
- 2) (empty list or set)

FLUSHALL

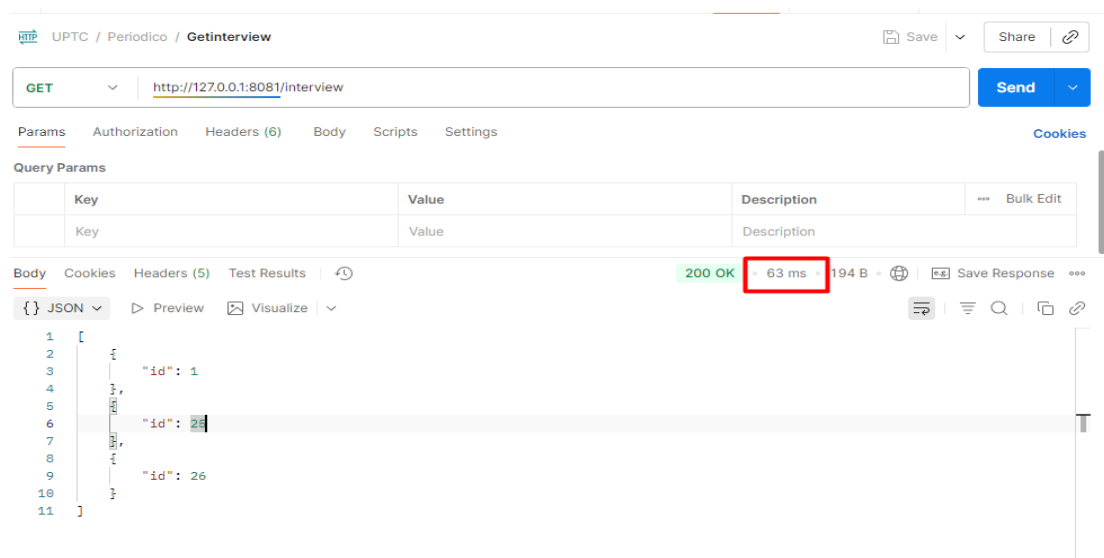
SCAN 0

- 1) "0"
- 2) 1) "newById::newByIdCache"
2) "allNew::newCache"

Validación

Para realizar la validación, se ejecutan nuestros **endpoints** utilizando herramientas como **Insomnia** o **Postman**, con el fin de efectuar las peticiones necesarias.

En la siguiente captura se observa que, en la primera ejecución del endpoint, la respuesta tarda **63 ms**, ya que aún no se encuentra almacenada en la **caché**.



En la segunda ejecución, se observa que el tiempo de respuesta disminuye a **5 ms**, una vez que la información ya se encuentra almacenada en la **caché**.

UPTC / Periodico / Getinterview

GET <http://127.0.0.1:8081/interview> Send

Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers (5) Test Results 200 OK 5 ms 194 B Save Response

{ } JSON Preview Visualize

```
1 [
2   {
3     "id": 1
4   },
5   {
6     "id": 25
7   },
8   {
9     "id": 26
10  }
11 ]
```

Para validar las llaves almacenadas en la **caché**, podemos utilizar la aplicación **Redis Insight**([Link de descarga oficial](#)), donde es posible visualizar el **nombre de la llave**, el **valor almacenado** y el **tiempo de vida (TTL)** asociado.

Total: 3 < 1 min Columns

STRING	newCache::newCache	48 s
STRING	interviewByldCache::allInterview	57 s
STRING	agencyCache::agencyCache	52 s

STRING interviewByldCache::allInterview Length: 219 TTL: 50

```
[
  "java.util.ArrayList",
  [
    {
      "@class": "com.uptc.frw.periodicoredis.model.Interview",
      "id": 1
    },
    {
      "@class": "com.uptc.frw.periodicoredis.model.Interview",
      "id": 25
    },
    {
      "@class": "com.uptc.frw.periodicoredis.model.Interview",
      "id": 26
    }
  ]
]
```